CMR UNIVERSITY

Private University Established in Karnataka State by Act No. 45 of 2013

CAPSTONE PROJECT PHASE I

CodeArchaeologist

An AST-Guided, LLM-Powered System for Legacy Code Comprehension and Automated Documentation

Vishal Kumar Jangir · 23BBTCS280
Syed Mudassir · 23BBTCS258
Junaid Pasha · 23BBTCS257

Guide
Mr. Sheik Jamil Ahmed
Department of CSE

School of Engineering and Technology CMR University 2026–27

Presentation outline

CMR UNIVERSITY

01 Problem statement and final objectives

02 Literature survey and identified gaps

03 System architecture and detailed design

04 Data-flow diagrams and use cases

05 Working model, verification, and remaining evaluation

Problem statement

Understanding an unfamiliar Python project requires tracing definitions, calls, and behavior across files.

01 Documentation may be absent or out of date.

02 Static structure helps navigation but does not fully explain behavior.

03 Generated explanations can misread source and need inspectable evidence.

04 A local tool should connect each finding or answer to the actual source.

Final project objectives

CMR UNIVERSITY

01 Extract Python definitions, imports, calls, and source locations.

02 Visualize module dependencies and supported function calls.

03 Report complexity, parse errors, and potentially unused symbols.

04 Generate documentation and source-linked local AI explanations.

05 Retrieve code by meaning and support natural-language questions.

Literature survey 1

Study	Contribution	Relevance to this project
CodeBERT Feng et al., 2020	Joint representations of code and natural language	Motivates semantic matching between questions and source.
Codex Chen et al., 2021	Code language models evaluated on programming tasks	Motivates language-model assistance and explicit error review.
GraphCodeBERT Guo et al., 2021	Incorporates data-flow structure into code representations	Supports examining structure when building code context.

Literature survey 2

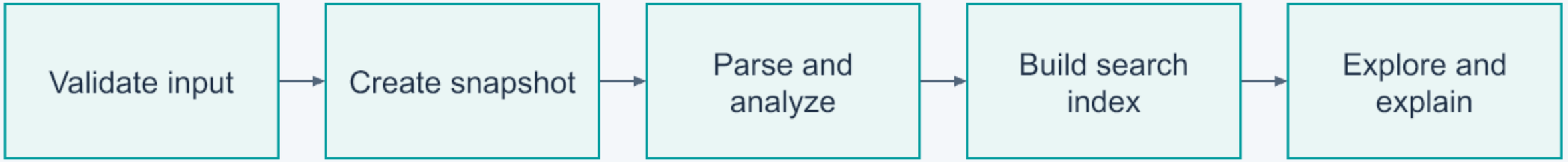
Study	Contribution	Relevance to this project
DeepCom Hu et al., 2018	Uses Java method structure for comment generation	Motivates extracting structured method information.
Transformer summarization Ahmad et al., 2020	Studies attention and relative code-token positions	Motivates focused summaries of bounded source context.
Big code survey Allamanis et al., 2018	Organizes probabilistic models and applications of code	Frames code learning and its evaluation challenges.

Identified gaps and challenges

Engineering challenge	Implemented response
Cross-file relationships are difficult to inspect	AST symbol records with dependency and call graphs.
Repository content exceeds a small model context	Retrieve bounded source chunks and relevant analysis.
Fluent answers may contain unsupported claims	Expose source lines and reject unknown source labels.
Dynamic Python calls are hard to resolve	Keep unresolved calls visible and label unused candidates.
Demonstration should run without a paid API	Local CPU generation and local embedding models.

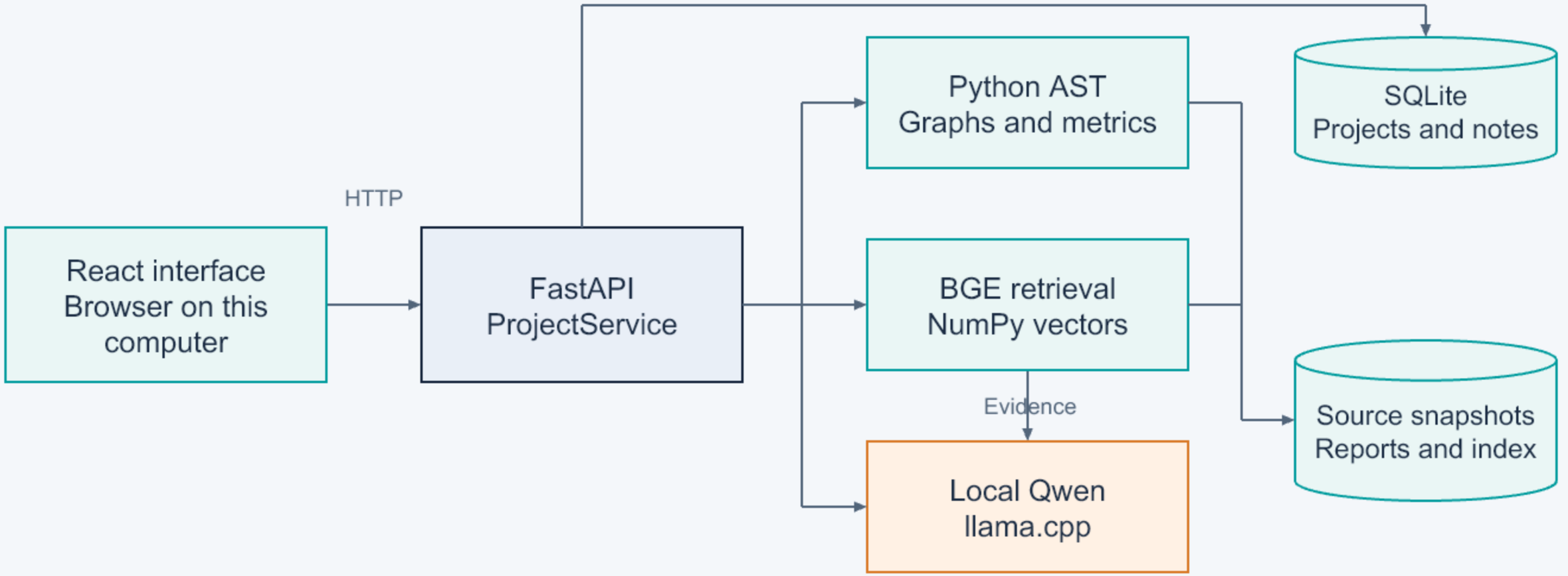
Methodology

Folder, ZIP, or demo



Static reports and source evidence remain available when generation is unavailable.

System architecture

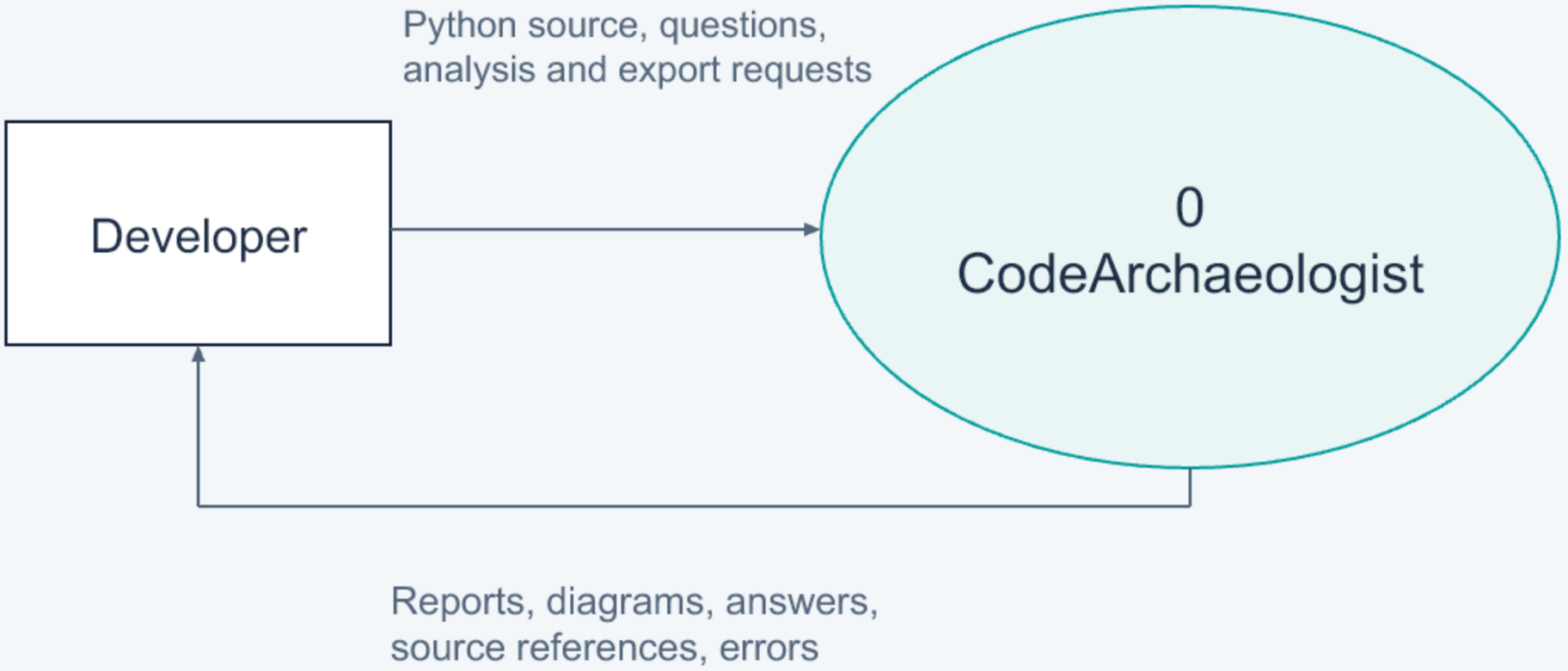


All application services and model inference run locally after first-time setup.

Detailed module design

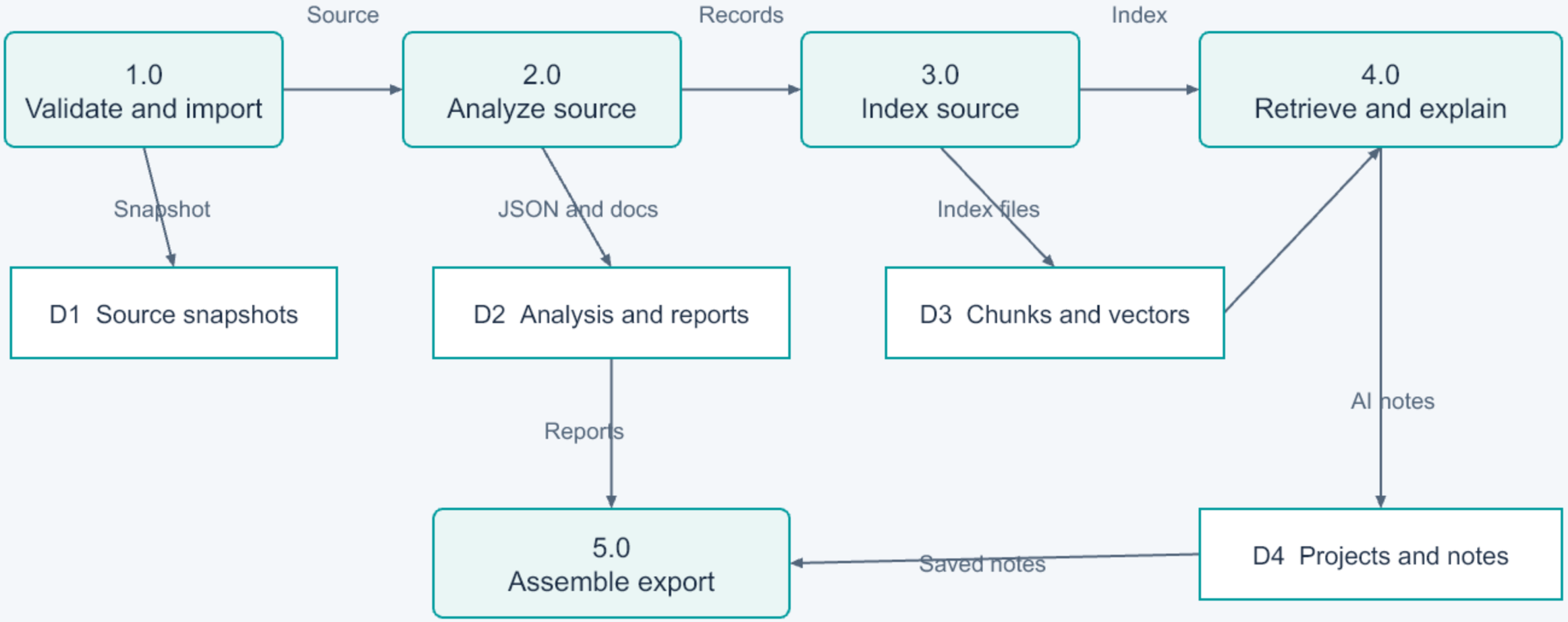
Subsystem	Main files	Output
Input and snapshots	ingestion.py, collector.py	Validated Python-only source snapshot
Static analysis	parser.py, analyzer.py, models.py	Symbols, graph edges, metrics, limitations
Retrieval	retrieval.py	Chunks, vectors, ranked source evidence
Generation	ai.py, service.py	Short facts, source labels, saved notes
Persistence and export	storage.py, reporter.py	SQLite records and downloadable reports
Application and UI	api.py, server.py, React	Local endpoints and interactive views

Data-flow diagram level 0



Context boundary includes the interface, backend, local models, and local storage.

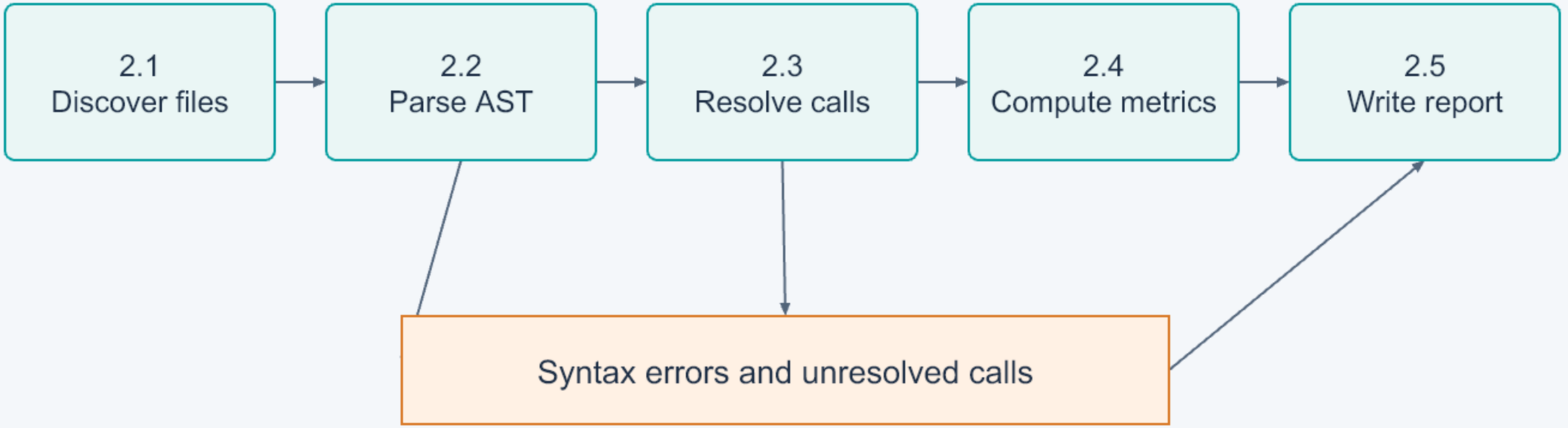
Data-flow diagram level 1



Developer provides source to 1.0, questions to 4.0, and export requests to 5.0.

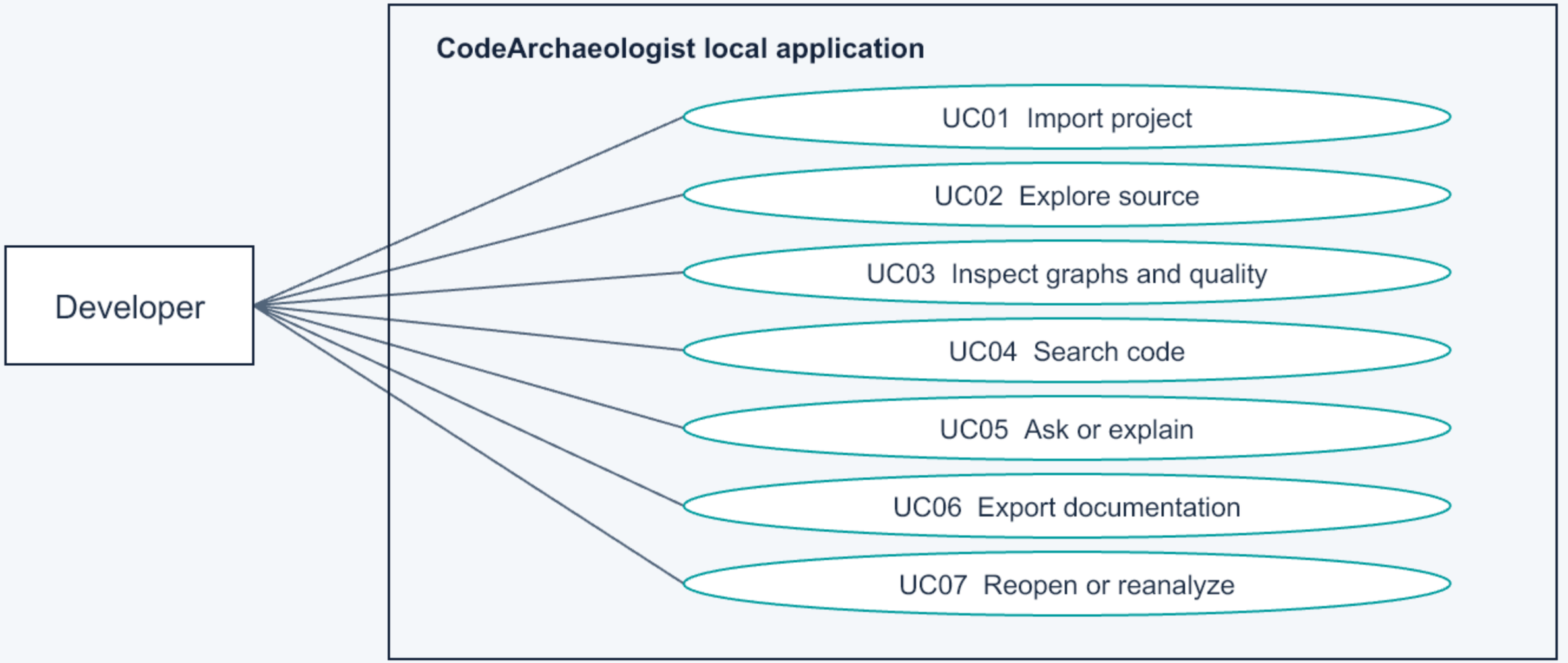
Data-flow diagram level 2 analysis

Input: D1 source snapshot



Output: D2 analysis.json and generated reports

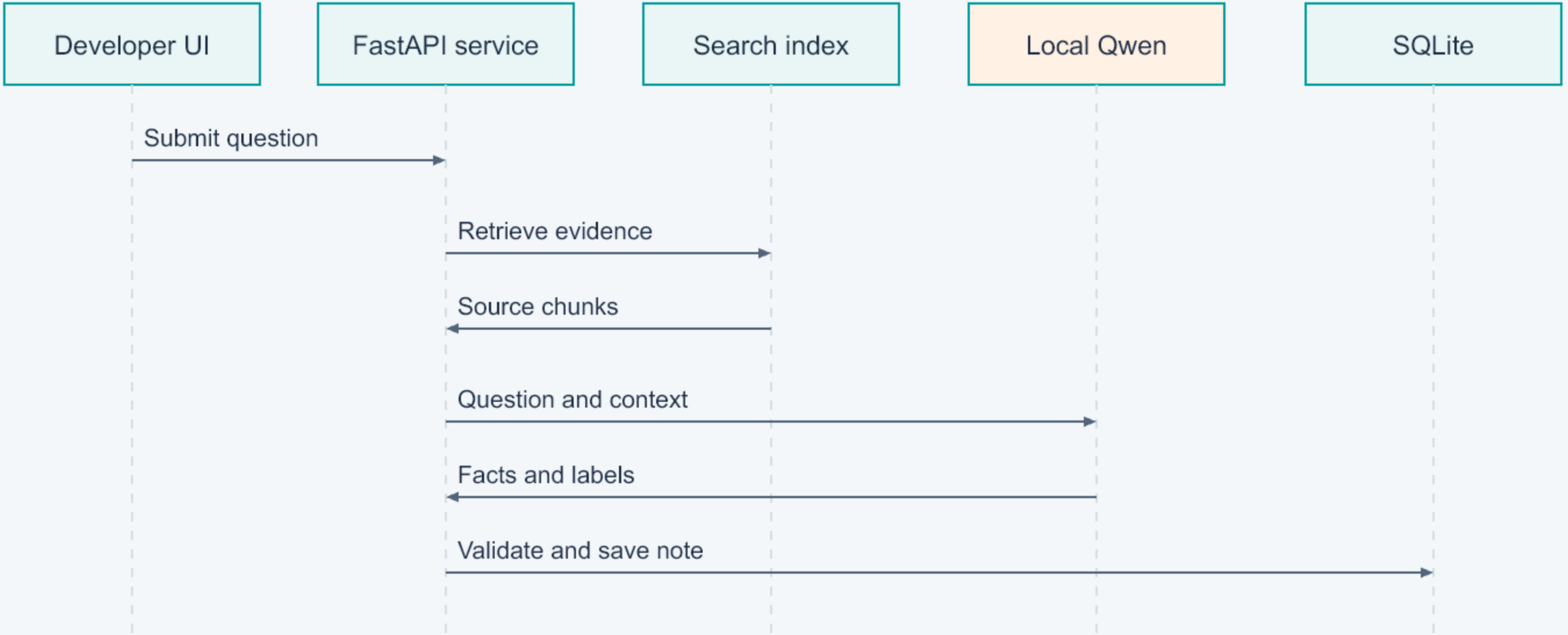
Use-case diagram



Use-case contracts

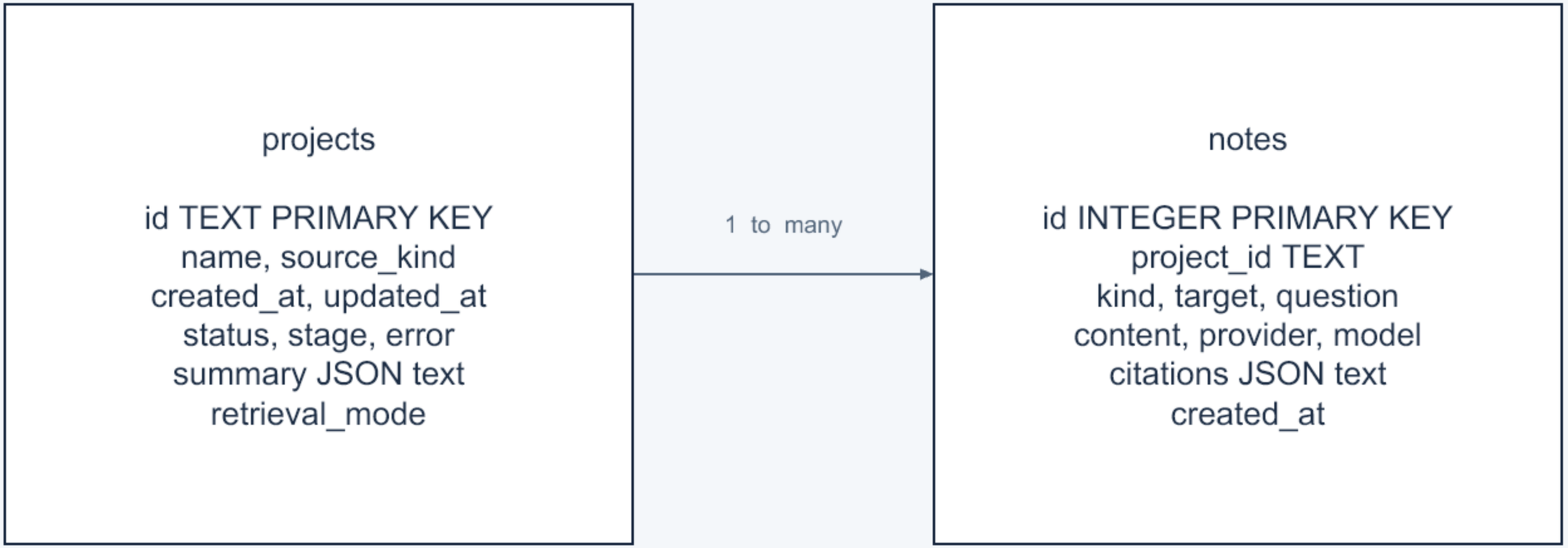
Use case	Precondition	Success and failure behavior
UC01 Import	Readable folder or valid ZIP	Create saved snapshot and report. Reject unsafe paths or limits.
UC05 Ask or explain	Ready project and available model	Retrieve evidence and save answer. Return error if model or source labels fail.
UC06 Export	Ready project	Download JSON, Markdown, SVG, offline HTML, and saved AI notes.
UC07 Reanalyze	Existing saved project	Rebuild the stored snapshot. Changed original source requires a new import.

Question-answer sequence



7 Return answer with source links to the interface

Data and storage design



Logical project link. The application manages this relationship.

Analysis output contract

```
schema_version  1.0.0
project         name, root, generated_at
summary         counts and average complexity
modules         definitions, imports, locations
call_graph      nodes and supported edges
quality         complex, long, and invalid code
unresolved_calls
dead_code       potentially unused candidates
limitations     interpretation boundaries
```

Static analysis is the shared contract for reports, retrieval, and the interface.

Static-analysis algorithms

CMR UNIVERSITY

01 Collect supported Python files and preserve package names.

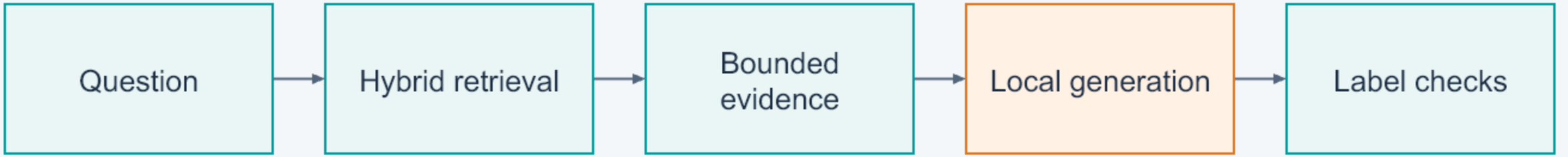
02 Extract definitions, imports, signatures, calls, and line ranges.

03 Resolve supported local, imported, self, and inferred-instance calls.

04 Estimate complexity and flag functions at score 8 or length 60 lines.

05 Keep unresolved calls and syntax errors available for review.

Retrieval and local generation



BGE-small-en-v1.5 384-dimensional vectors
Qwen2.5-Coder-1.5B-Instruct Q2_K CPU inference
Valid source labels help inspection. They do not prove each sentence is correct.

Implemented technology stack

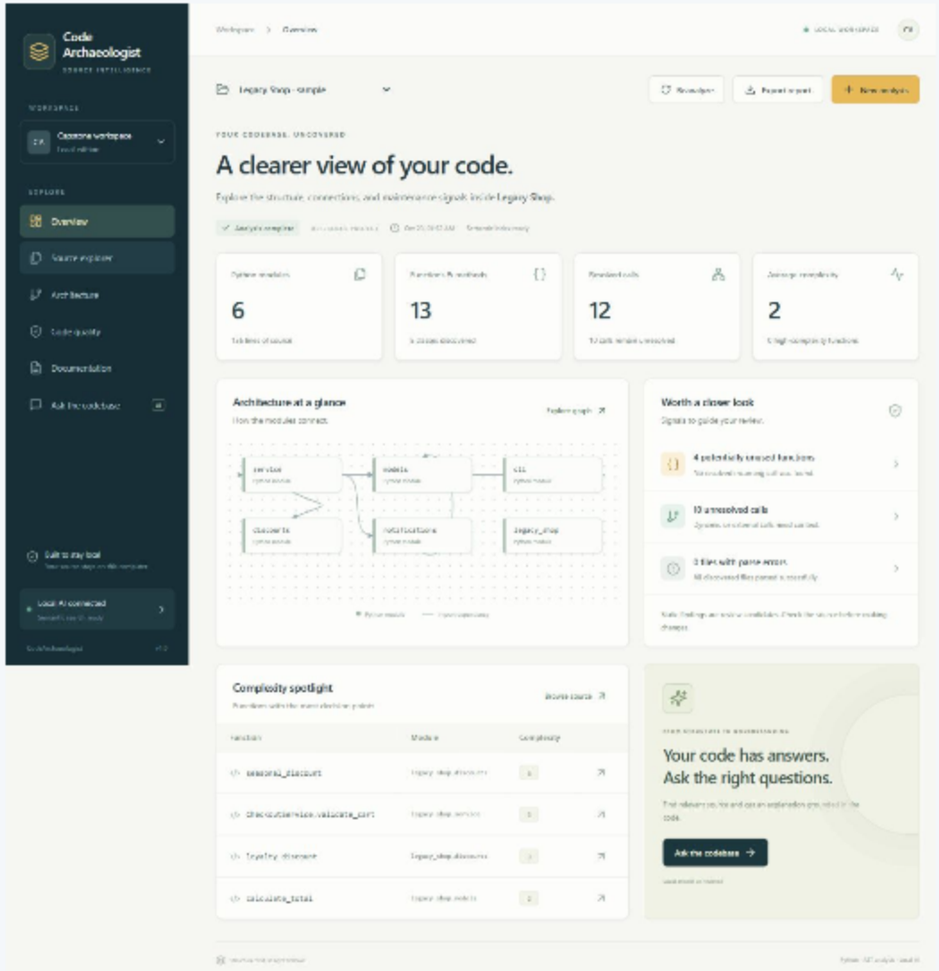
Layer	Delivered implementation
Parser and graph analysis	Python ast, typed records, custom resolver and edge lists
Web application	FastAPI, Uvicorn, React, local SVG graph views
Persistence	SQLite, immutable source snapshots, JSON and Markdown reports
Semantic search	FastEmbed, BGE-small-en-v1.5, ONNX Runtime, NumPy
Generation	Qwen2.5-Coder-1.5B-Instruct Q2_K with llama.cpp
Packaging	Windows launchers, pinned Python dependencies, prebuilt UI

Working application

Legacy Shop reference project

6 modules
13 functions and methods
135 source lines
12 resolved call edges
0 parse errors

Measured fixture results, not a general accuracy benchmark.



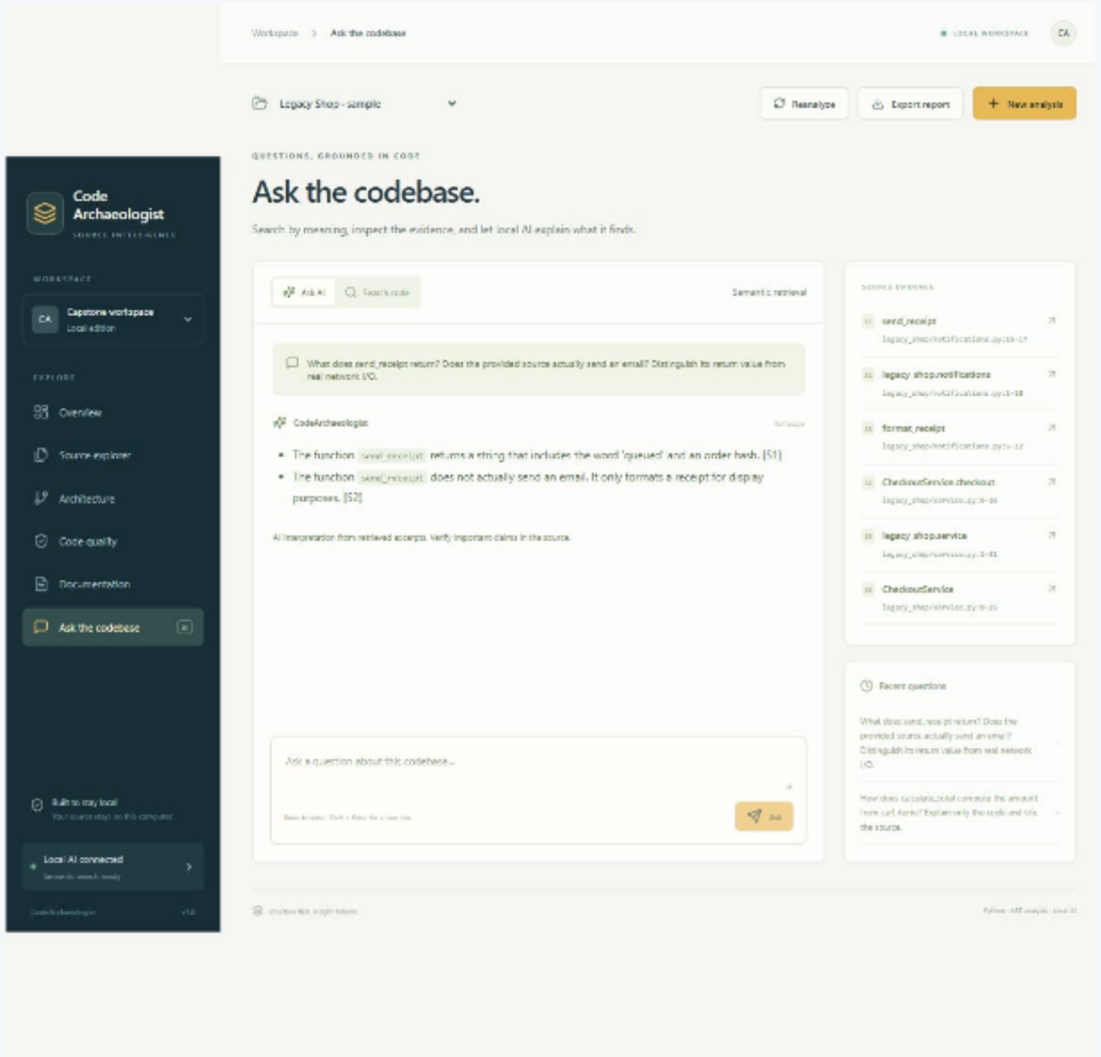
Local AI with inspectable source

Real local model response

The recorded calculation answer took 24.7 seconds on this computer.

Answers and citations still need source review.

No paid API or model training is required.



Verification results

Check	Observed result	Scope
Automated tests	15 passed	Parser, CLI, API, input handling, persistence, AI protocol
Exact expected symbol in top 3	2 of 4 questions	Small Legacy Shop fixture
Relevant source excerpt in top 3	4 of 4 questions	Module excerpt may cover the expected function
Recorded AI question latency	24.73 seconds	One CPU inference example
Application self-analysis	22 modules, 118 functions	Saved implementation snapshot, no parse errors

Limitations and future work

CMR UNIVERSITY

01 Dynamic Python behavior leaves some calls unresolved.

02 Unused-code and complexity findings require developer review.

03 Small-model answers can be incorrect or cite the wrong excerpt.

04 Broader repository testing and accuracy evaluation remain.

05 Other languages, Git URL import, accounts, and live watching are future work.

Project status and presentation roles

Area	Current status
Analysis foundation	Implemented and covered by regression tests
Local AI and semantic interface	Implemented and exercised with actual local models
Broader evaluation	Remaining work on additional repositories and reviewed answers
Vishal Kumar Jangir	Suggested sections: problem, objectives, literature
Syed Mudassir	Suggested sections: architecture, DFDs, use cases, data design
Junaid Pasha	Suggested sections: implementation, demo, results, limitations

References

[1] Feng et al. CodeBERT. Findings of EMNLP, 1536–1547, 2020.

[2] Chen et al. Evaluating Large Language Models Trained on Code. arXiv:2107.03374, 2021.

[3] Guo et al. GraphCodeBERT. ICLR, 2021. arXiv:2009.08366.

[4] Hu et al. Deep Code Comment Generation. ICPC, 200–210, 2018.

[5] Ahmad et al. A Transformer-based Approach for Source Code Summarization. ACL, 4998–5007, 2020.

[6] Allamanis et al. A Survey of Machine Learning for Big Code and Naturalness. ACM Computing Surveys, 2018.

CodeArchaeologist

Questions and live demonstration

Guide Mr. Sheik Jamil Ahmed
Vishal Kumar Jangir Syed Mudassir Junaid Pasha

CMR University School of Engineering and Technology 2026–27